Machine Learning and Pattern Recognition

Linear Regression

Dr. Kewei Li

Department of Digital Engineering
Long Island University

March 2, 2025

Introduction

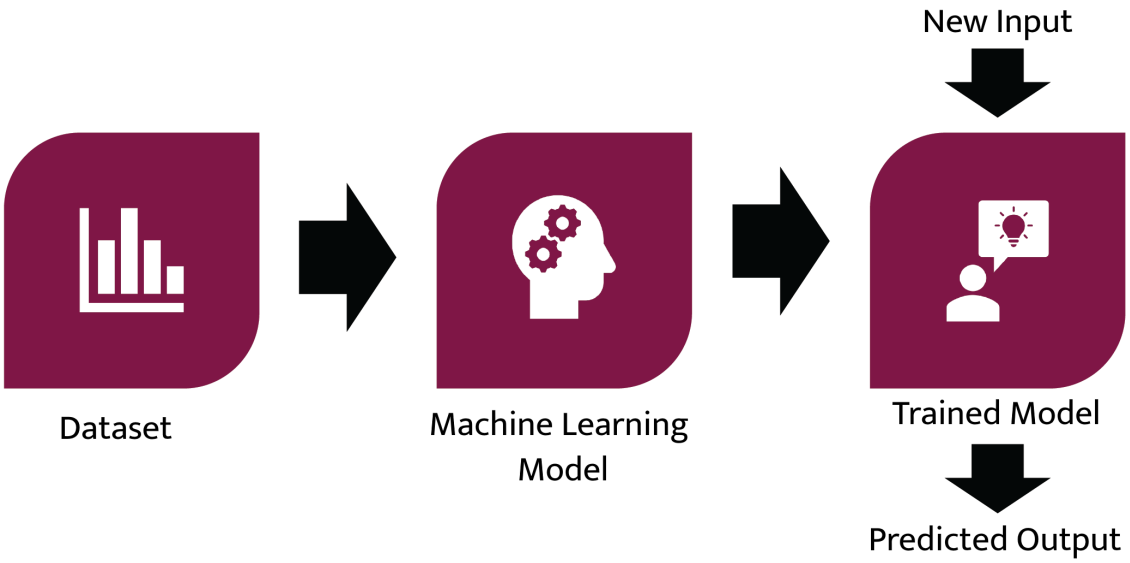
- Introduction
- Linear Regression
- Gradient Descent
- Bias-Variance Tradeoff

Outline

- Introduction
- Linear Regression
- Gradient Descent
- Bias-Variance Tradeoff

Introduction

Recap: Supervised learning



Linear regression: a basic example of supervised learning

Introduction

Linear regression captures many fundamental concepts in ML

- Function approximation view of machine learning
- Optimization and gradient descent
- Regularization
- Training/validation/test split
- Bias-variance tradeoff

Introduction

Linear Regression Problem

- A hypothesis, employed as a model for solving the regression problem, maps inputs to outputs,

$$\mathbf{x} \to \boxed{h} \to y$$

- $\mathbf{x} \in \mathbb{R}^d$ is called an input or features
- $h : \mathbb{R}^d \to \mathbb{R}$ is called hypothesis or model function
- $y \in \mathbb{R}$ is called the output, label, target, or response

Introduction

Linear Functions

- Consider the space of linear functions $h(\mathbf{x})$ defined by

$$h(\mathbf{x}) = \boldsymbol{\theta}^{\mathrm{T}}\mathbf{x} = \left[\theta_1, \theta_2, \cdots, \theta_d\right] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} = \theta_1 x_1 + \cdots + \theta_d x_d$$

- $\boldsymbol{\theta} \in \mathbb{R}^d$ is called the parameters or parameter vector

Introduction

Linear Regression Problem

- Given a dataset $\mathcal{D}_{\text{train}} = \left\{\left(\mathbf{x}^{(1)}, y^{(1)}\right), \cdots, \left(\mathbf{x}^{(n)}, y^{(n)}\right)\right\}$ of n data points and a hypothesis $h(\mathbf{x}; \boldsymbol{\theta})$
- Goal: Estimate the parameters $\boldsymbol{\theta}$ of the hypothesis $h(\mathbf{x}; \boldsymbol{\theta})$ such that $y \approx h(\mathbf{x}; \boldsymbol{\theta})$ for any new input $\mathbf{x}$
- The training error of $h(\mathbf{x}; \boldsymbol{\theta})$ can be defined as the average loss on the training data:

$$\mathcal{E}_{\text{train}}(h; \boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(h(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$$

$\mathcal{L}(\cdot)$: a loss function measures how good is our prediction $h(\mathbf{x}^{(i)}; \boldsymbol{\theta})$ for the data point $(x^{(i)}, y^{(i)})$.

Introduction

Linear Regression Problem

- The training error indicates how good is our hypothesis when characterizing the relationship between $\mathbf{x}$ and y
- However, what we care about most is error during testing,

$$\mathcal{E}_{\text{test}}(h) = \frac{1}{n'} \sum_{i=n+1}^{n+n'} \mathcal{L}(h(\mathbf{x}^{(i)}), y^{(i)}),$$

  on n' of new data points that were not used during the training
- Task: find a hypothesis with small training error, and it also generalizes well to new data (a small test error)

Linear Regression

- Given dataset $\mathcal{D}$, a loss function $\mathcal{L}$, and a hypothesis class $\mathcal{H}$, we need a method to identify an optimal hypothesis in the class.
- A general way to approach this problem is by framing the linear regression problem as an optimization problem
- We define an objective function, $J(\theta)$ or $J(\theta; \mathcal{D})$ to indicate the dependence of J on data
- Find parameter values that minimize the objective function:

$$\theta^* = \arg\min_{\theta} J(\theta)$$

Linear Regression

- In general, there is no guarantee that there exists a unique set of parameters which minimize the objective function
- A common form of objective function is:

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \underbrace{\mathcal{L}(h(\mathbf{x}^{(i)};\theta), y^{(i)})}_{\text{loss}} + \lambda R(\theta).$$

  λ is a non-negative constant, and $R(\theta)$ is regularizer
- Minimizing the loss improves prediction accuracy.

Introduction

- An example regression problem is the polynomial fitting problem we studied in Lecture 2
- The simplest form of linear regression models are also linear functions of the input variables
- A more useful class of functions is obtained by taking linear combinations of a fixed set of nonlinear functions

Outline

- Introduction
- Linear Regression
- Gradient Descent
- Bias-Variance Tradeoff

Linear Regression

- Consider a class of hypotheses $\mathcal{H}$ that might provide a set of possible models for the relationship between $\mathbf{x}$ and y in our data.
- We start with a very simple class of linear hypotheses:

$$h(\mathbf{x}; \boldsymbol{\theta}, \theta_0) = \theta_0 + \boldsymbol{\theta}^{\mathrm{T}}\mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$$

- The parameter θ_0 allows for any fixed offset in the data and is called a bias or intercept parameter.
- In one dimension ($d = 1$), this becomes a line. In two dimensions ($d = 2$), it is a plane. In higher dimensions, it describes a hyperplane

Linear Regression

- We can extend the class of hypotheses by considering linear combinations of nonlinear functions of the input variables,

$$h(\mathbf{x}; \boldsymbol{\theta}, \theta_0) = \theta_0 + \sum_{j=1}^{d} \theta_j \phi_j(\mathbf{x})$$

  where $\phi_j(\mathbf{x})$ are known as basis functions
- If $\phi_j(\mathbf{x}) = x_j$, we recover the linear hypotheses.

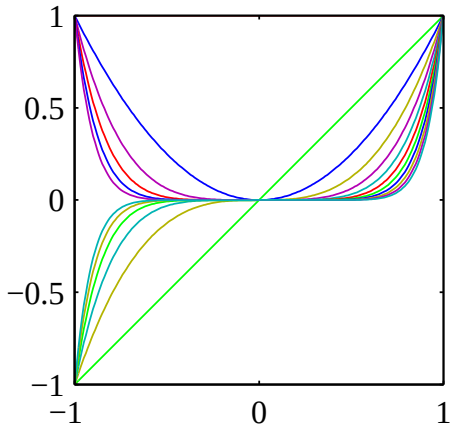
Linear Regression

- If we define a dummy "basis function" $\phi_0(x) = 1$, then

$$h(\mathbf{x}; \boldsymbol{\theta}) = \sum_{j=0}^{d+1} \theta_j \phi_j(\mathbf{x}) = \boldsymbol{\theta}^{\mathrm{T}} \boldsymbol{\phi}(\mathbf{x})$$

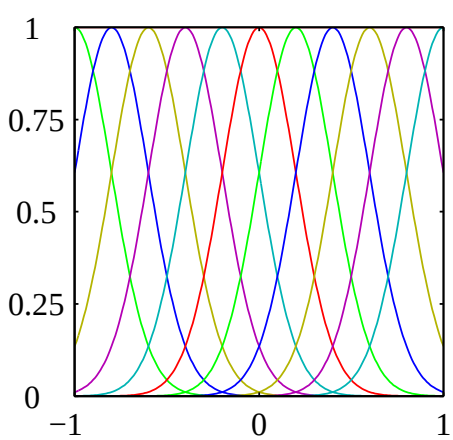
- By using nonlinear basis functions, the function $h(\mathbf{x}; \boldsymbol{\theta})$ becomes a nonlinear function of $\mathbf{x}$
- Functions of this form are called linear models.
- The linearity in $\boldsymbol{\theta}$ will greatly simplify the analysis of this class of models. However, it also leads to some significant limitations

Linear Regression



- The polynomial fitting in Lecture 2 is a particular example of the linear model
- The basis functions take the form as powers of x so that $\phi_j(x) = x^j$

Linear Regression

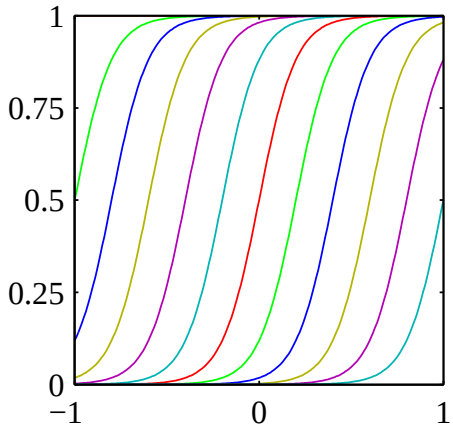


- There are many other types of basis functions.
- For example, the "Gaussian" basis functions,

$$\phi_j(x) = \exp\left[-\frac{(x-\mu_j)^2}{2s^2}\right]$$

- The parameters μ_j govern the locations of the basis functions in the input space, and s governs their spatial scale

Linear Regression



- Another option is the sigmoidal basis function

$$\phi_j(x) = \sigma\left(\frac{x - \mu_j}{s}\right)$$

where $\sigma(a)$ is the logistic sigmoid function defined by

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

- Equivalently, we can use the $\tanh$ function because $\tanh(a) = 2\sigma(a) - 1$

Linear Regression

- Our objective in linear regression is to find a hypothesis that works well on our training data.
- We define a loss function to describe the quality of the predictions our hypothesis is making, when compared to the "target" in the data set
- The choice of loss function is part of modeling your domain.
- In the absence of additional information about the problem, we typically use squared loss:

$$\mathcal{L}(g, a) = (g - a)^2$$

g is a "guess" or prediction by the hypothesis, and a is the "actual" observation ($a = y$)

Linear Regression

- With the choice of squared loss, the average loss becomes the so-called mean squared error (MSE)

$$\mathcal{E}_{\text{train}}(h;\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(h(\mathbf{x}^{(i)};\boldsymbol{\theta}), y^{(i)}) = \frac{1}{n}\sum_{i=1}^{n}\left(h(\mathbf{x}^{(i)};\boldsymbol{\theta}) - y^{(i)}\right)^2$$

- With the linear hypothesis and squared loss, our objective is to find parameters that minimize the MSE

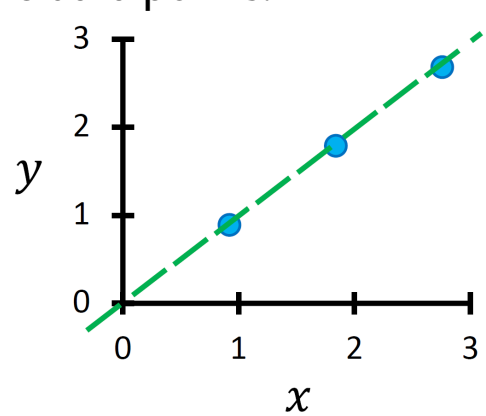
$$J(\boldsymbol{\theta},\theta_0) = \frac{1}{n}\sum_{i=1}^{n}\left(\theta_0 + \boldsymbol{\theta}^{\mathrm{T}}\mathbf{x}^{(i)} - y^{(i)}\right)^2$$

Linear Regression

- Minimization of the objective function yields the solution

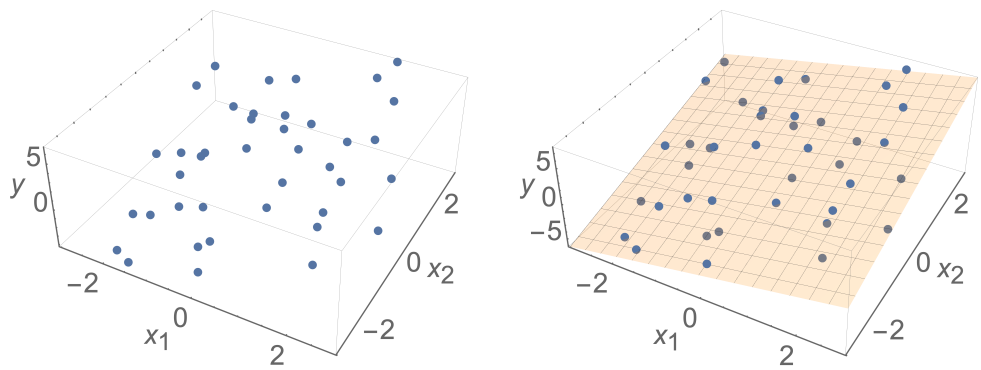
$$\boldsymbol{\theta}^*, \theta_0^* = \arg\min_{\boldsymbol{\theta},\theta_0} J(\boldsymbol{\theta},\theta_0)$$

- For one-dimensional data ($d = 1$), this corresponds to fitting a line ($h(x) = ax + b$) to some data points.



Linear Regression

- For higher dimensions, the hypothesis represents a d-dimensional hyperplane embedded in a $(d+1)$-dimensional space (d plus the y dimension)



Linear Regression

Given the objective, how can we find a good set of parameters?

We can guess a large number (k) of different values of $\boldsymbol{\theta}$ and θ_0, return the one with the smallest error

Algorithm 1 Random-Regression

Require: Training dataset $\mathcal{D}$, integer k

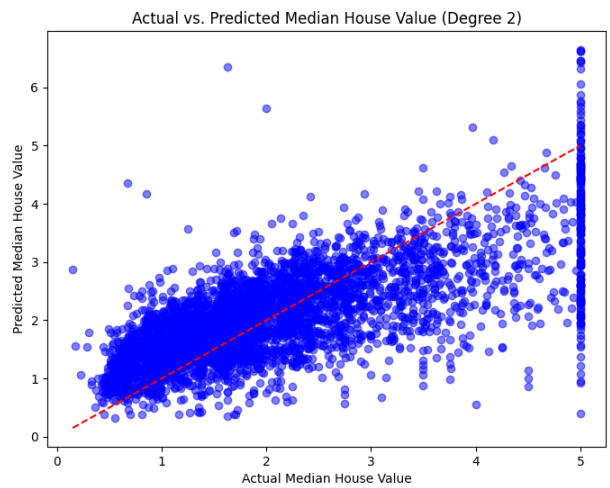
1: **for** $i = 1$ to k **do**
2: Randomly generate parameters $\boldsymbol{\theta}^{(i)}, \theta_0^{(i)}$
3: **end for**
4: Let $i = \arg\min_j J(\boldsymbol{\theta}^{(j)}, \theta_0^{(j)})$
5: **return** $\boldsymbol{\theta}^{(i)}, \theta_0^{(i)}$

Linear Regression: Example Implementation in Python

The California Housing Dataset in Scikit-learn includes features such as:

- MedInc: Median income in block group
- HouseAge: Median house age in block group
- AveBedrms: Average number of bedrooms per household

Goal: Predict the target variable "Median house value"



Analytical Solution

- Instead of finding a linear hypothesis that minimizes mean squared error, we can also find a closed-form solution
- This general problem is often called the ordinary least squares (ols)
- At first, we ignore the offset θ_0. Suppose for now, we have,

$$y = \boldsymbol{\theta}^{\mathrm{T}}\mathbf{x}$$

- In this case, the objective becomes

$$J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\left(\boldsymbol{\theta}^{\mathrm{T}}\mathbf{x}^{(i)} - y^{(i)}\right)^2$$

Analytical Solution

It is possible to approach this problem by

- Finding $\partial J / \partial \theta_k$ for k in $\{1, 2, \cdots, d\}$
- Constructing a set of equations by setting $\partial J / \partial \theta_k = 0$, and
- Solving the system of equations for θ_k
- Additional steps: to check that the result is a minimum (rather than a maximum or an inflection point)

Analytical Solution

- To apply techniques like this to more complex problems, we will work with matrix notation
- In particular, the gradient of J with respect to $\boldsymbol{\theta}$ is a column vector

$$\nabla_{\boldsymbol{\theta}} J = \begin{bmatrix} \partial J / \partial \theta_1 \\ \partial J / \partial \theta_2 \\ \vdots \\ \partial J / \partial \theta_d \end{bmatrix}$$

Analytical Solution

- We write training data as matrices $\mathbf{X}$ and $\mathbf{Y}$. Each row of $\mathbf{X}$ is an input, and each row of $\mathbf{Y}$ is the corresponding target output

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

- Now we can write

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} (\boldsymbol{\theta}^{\mathrm{T}} \mathbf{x}^{(i)} - y^{(i)})^2 = \frac{1}{n} (\mathbf{X}\boldsymbol{\theta} - \mathbf{Y})^{\mathrm{T}} (\mathbf{X}\boldsymbol{\theta} - \mathbf{Y}).$$

Analytical Solution

- Using Matrix/Vector Calculus, we obtain,

$$\begin{aligned} \nabla_{\theta} J(\boldsymbol{\theta}) &= \frac{1}{n} \nabla_{\boldsymbol{\theta}} \left[(\mathbf{X}\boldsymbol{\theta})^{\mathrm{T}} \mathbf{X}\boldsymbol{\theta} - \mathbf{Y}^{\mathrm{T}} \mathbf{X}\boldsymbol{\theta} - (\mathbf{X}\boldsymbol{\theta})^{\mathrm{T}} \mathbf{Y} + \mathbf{Y}^{\mathrm{T}} \mathbf{Y} \right] \\ &= \frac{2}{n} \left(\mathbf{X}^{\mathrm{T}} \mathbf{X}\boldsymbol{\theta} - \mathbf{X}^{\mathrm{T}} \mathbf{Y} \right). \end{aligned}$$

- Setting this equal to zero and solving for $\boldsymbol{\theta}$ yields the final closed-form solution:

$$\boldsymbol{\theta}^* = \left(\mathbf{X}^{\mathrm{T}} \mathbf{X} \right)^{-1} \mathbf{X}^{\mathrm{T}} \mathbf{Y}$$

- So, given our data, we can directly compute the linear regression that minimizes mean squared error.

Analytical Solution

- Now, how do we deal with the offset θ_0?
- We augment the original input vector with a "fake" element of 1 and add a corresponding parameter to $\boldsymbol{\theta}$ vector
- We define columns vectors such that,

$$\mathbf{x}_{\text{aug}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \\ 1 \end{bmatrix}, \quad \boldsymbol{\theta}_{\text{aug}} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_d \\ \theta_0 \end{bmatrix}$$

where "aug" denotes that it has been augmented.

Analytical Solution

- Then we can now write the linear hypothesis as if there is no offset,

$$h(\mathbf{x}_{\text{aug}}; \boldsymbol{\theta}_{\text{aug}}) = \boldsymbol{\theta}_{\text{aug}}^{\mathrm{T}} \mathbf{x}_{\text{aug}}$$

- We apply this concept to all data points to form the augmented data matrix

$$\mathbf{X}_{\text{aug}} = \begin{bmatrix} x_1^{(1)} & \dots & x_d^{(1)} & 1 \\ \vdots & \ddots & \vdots & \vdots \\ x_1^{(n)} & \dots & x_d^{(n)} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{X} & \mathbb{1} \end{bmatrix}$$

where $\mathbb{1}$ is an n-by-1 vector of all ones.

- Then, we can use the closed-form solution to find $\boldsymbol{\theta}_{\text{aug}}$ that minimizes the mean squared error

Data Centering

- In fact, augmenting a "fake" feature of 1 to the input vector, is also useful for an important idea: centering
- Centering eliminates the need for fitting an intercept, and thereby offers an alternative way to avoid dealing with θ_0 directly.
- By centering, we mean subtracting the average (mean) of each feature from all data points, and we apply the same operation to the labels.

Data Centering

- For an example of a dataset before and after centering

	Temperature	Population	Pollution
Chicago	90	45	7.2
New York	20	32	9.5
Boston	35	100	8.4

$$\Downarrow$$

	Temperature	Population	Pollution
Chicago	41.66	-14	-1.66
New York	-28.33	-27	1.133
Boston	-13.33	41	0.033

Data Centering

- With a centered dataset, even if we were to search for an offset θ_0, it would naturally fall out to be 0.
- To prove this mathematically, first, for a centered dataset, two claims immediately follow:
  - Each column of $\mathbf{X}$ sums up to zero, that is, $\mathbf{X}^{\mathrm{T}}\mathbb{1} = \mathbb{1}^{\mathrm{T}}\mathbf{X} = 0$
  - Similarly, the mean of the labels is 0, so $\mathbf{Y}^{\mathrm{T}}\mathbb{1} = \mathbb{1}^{\mathrm{T}}\mathbf{Y} = 0$

Data Centering

- Recall that our goal is to find an optimal hypothesis, parameterized by $\boldsymbol{\theta}$ and θ_0.
- Substituting $\mathbf{X}_{\text{aug}}$ in to the closed-form solution for $\boldsymbol{\theta}^*_{\text{aug}}$

$$\begin{aligned}
\boldsymbol{\theta}^*_{\text{aug}} &= \left(\begin{bmatrix}\mathbf{X}^T \\ \mathbb{1}^T\end{bmatrix}\begin{bmatrix}\mathbf{X} & \mathbb{1}\end{bmatrix}\right)^{-1}\begin{bmatrix}\mathbf{X}^T \\ \mathbb{1}^T\end{bmatrix}\mathbf{Y} = \begin{bmatrix}\mathbf{X}^T\mathbf{X} & \mathbf{X}^T\mathbb{1} \\ \mathbb{1}^T X & \mathbb{1}^T\mathbb{1}\end{bmatrix}^{-1}\begin{bmatrix}\mathbf{X}^T \\ \mathbb{1}^T\end{bmatrix}\mathbf{Y} \\
&= \begin{bmatrix}\mathbf{X}^T\mathbf{X} & 0 \\ 0 & n\end{bmatrix}^{-1}\begin{bmatrix}\mathbf{X}^T \\ \mathbb{1}^T\end{bmatrix}\mathbf{Y} = \begin{bmatrix}(\mathbf{X}^T X)^{-1}\mathbf{X}^T\mathbf{Y} \\ n\mathbb{1}^T\mathbf{Y}\end{bmatrix} \\
&= \begin{bmatrix}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} \\ 0\end{bmatrix} = \begin{bmatrix}\boldsymbol{\theta}^* \\ \theta_0^*\end{bmatrix}
\end{aligned}$$

Outline

- Introduction
- Linear Regression
- Gradient Descent
- Bias-Variance Tradeoff

Gradient Descent

- We presented an objective function for linear regression problem.
- But we need a way to find the optimal solution $\Theta^* = \arg\min_\Theta J(\Theta)$, particularly when the objective function is complex
- There is an enormous literature on the mathematical and algorithmic foundations of optimization
- For this class, we will consider one of the simplest methods, called gradient descent.

Gradient Descent

- Intuitively, in one or two dimensions, we can easily think of $J(\theta)$ as defining a surface over θ; that same idea extends to higher dimensions.
- Now, our objective is to find the θ at the lowest point on that surface.
- The idea of gradient descent: you start at some arbitrary point on the surface, look to see in which direction the "hill" goes down most steeply.
- Take a small step in that direction, determine the direction of steepest descent from where you are, take another small step, etc.

Gradient Descent

- We start by considering gradient descent in one dimension ($d = 1$)
- Assume $\theta \in \mathbb{R}$, and that we know both $J(\theta)$ and its first derivative with respect to θ
- To apply gradient descent algorithm on an arbitrary function f, we need its gradient $\nabla_\theta f$ to be well defined.
- We introduce some hyper-parameters:
  - The initial value for parameter θ
  - A step-size η, also called learning rate
  - An accuracy parameter ε

Gradient Descent

Pseudo-code for gradient descent with a constant η

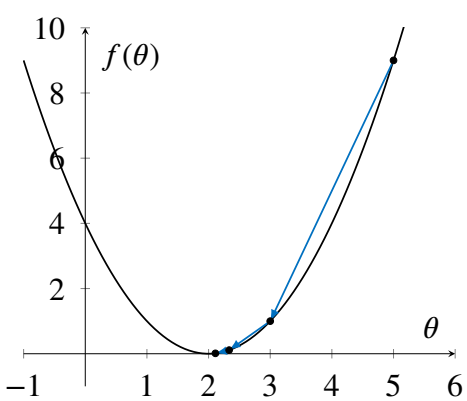
Algorithm 2 Gradient-Descent for 1D function

```
procedure GRADIENT-DESCENT-1D(θ_init, η, f, f', ε)
    θ^(0) ← θ_init
    t ← 0
    repeat
        t ← t + 1
        θ^(t) = θ^(t−1) − η f'(θ^(t−1))
    until |f'(θ^(t))| < ε
end procedure
```

This algorithm terminates when the derivative of the function f is sufficiently small

Gradient Descent

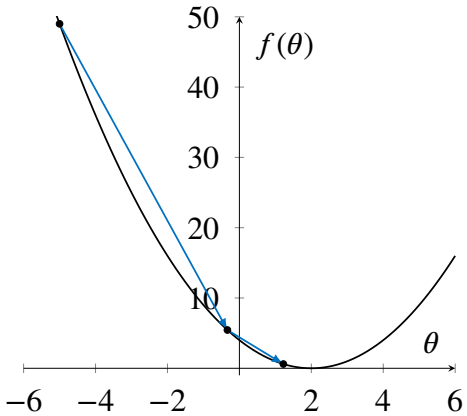
Example: Function $f(\theta) = (\theta - 2)^2$



- If we start gradient descent at $\theta_{\text{init}} = 5.0$ with $\eta = 1/3$.
- At $t = 0$, $\theta^{(0)} = 5.0$
- At $t = 1$, $f'(\theta^{(0)}) = 2(\theta^{(0)} - 2) = 6$.
  $\theta^{(1)} = \theta^{(0)} - \eta f'(\theta^{(0)}) = 3$
- At $t = 2$, $f'(\theta^{(1)}) = 2(\theta^{(1)} - 2) = 2$.
  $\theta^{(2)} = \theta^{(1)} - \eta f'(\theta^{(1)}) = 7/3$
- At $t = 3$, $f'(\theta^{(2)}) = 2(\theta^{(2)} - 2) = 2/3$.
  $\theta^{(3)} = \theta^{(2)} - \eta f'(\theta^{(2)}) = 19/9$

Gradient Descent

Example: Function $f(\theta) = (\theta - 2)^2$



- If we start gradient descent at $\theta_{\text{init}} = -5.0$ with $\eta = 1/3$.
- At $t = 0$, $\theta^{(0)} = -5.0$
- At $t = 1$, $f'(\theta^{(0)}) = 2(\theta^{(0)} - 2) = -14$. $\theta^{(1)} = \theta^{(0)} - \eta f'(\theta^{(0)}) = -1/3$
- At $t = 2$, $f'(\theta^{(1)}) = 2(\theta^{(1)} - 2) = -14/3$. $\theta^{(2)} = \theta^{(1)} - \eta f'(\theta^{(1)}) = 11/9$
- What happends if η is smaller?
- What happends if θ_{init} is far from the solutoin?

Gradient Descent

There are many other reasonable ways to terminate, including:

- Stop after a fixed number of iterations T, i.e., when $t = T$. Practically, this is the most common choice.
- Stop when the change in the value of the parameter θ is sufficiently small, i.e., when $\left|\theta^{(t)} - \theta^{(t-1)}\right| < \varepsilon$

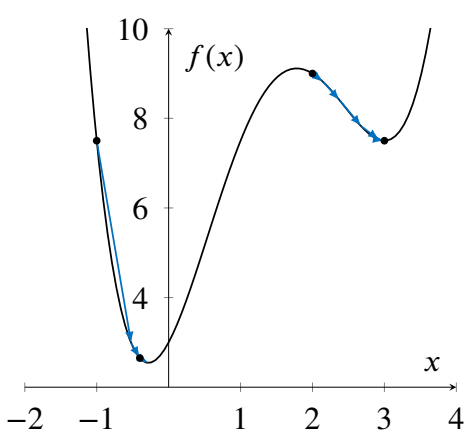
Gradient Descent

Theorem
Choose any small distance $\tilde{\varepsilon} > 0$. If we assume that f has a minimum, is sufficiently "smooth" and convex, and if the learning rate η is sufficiently small, gradient descent will reach a point within of a global optimum point.

- However, we must be careful when choosing the learning rate to prevent slow convergence, non-converging oscillation around the minimum, or divergence.

Gradient Descent

- If f is non-convex, where gradient descent converges to depends on the initial guess of x_{init}
- The plot below shows how gradient descent started from each point heads toward two different local optimum points



Gradient Descent

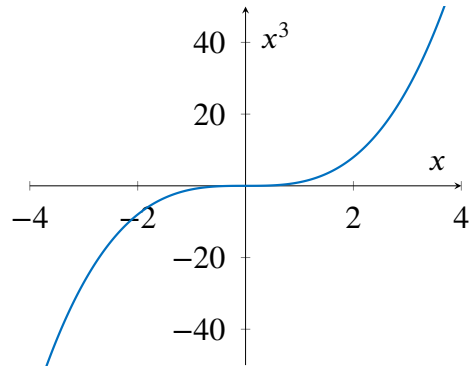
- Let f be a real-valued function defined over some domain D
- A point $x_0 \in D$ is called a global minimum point of f if $f(x_0) \leq f(x)$ for all other $x \in D$
- A point $x_0 \in D$ is called a local minimum point of a function f if there exists some constant $\varepsilon > 0$ such that for all x within the interval defined by $d(x, x_0) < \varepsilon$, $f(x_0) \leq f(x)$, where d is some distance metric, e.g., $d(x, x_0) = ||x - x_0||$
- A global minimum point is also a local minimum point, but a local minimum point does not have to be a global minimum point.

Gradient Descent

- If f is non-convex (and sufficiently smooth), one expects that gradient descent (run long enough with small enough learning rate) will get very close to a point at which the gradient is zero
- Although we cannot guarantee that it will converge to a global minimum point.
- There are two notable exceptions.
- First, gradient descent can get stagnated while approaching a point which is not a local minimum or maximum, but satisfies $f'(x) = 0$.

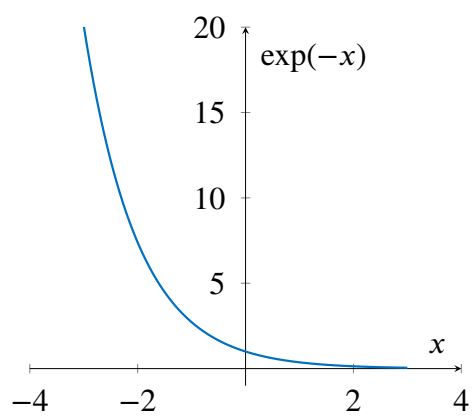
Gradient Descent

- For example, for $f(x) = x^3$, starting gradient descent from the initial guess $x_{\text{init}} = 1$, while using learning rate $\eta < 1/3$ will lead to $x^{(k)}$ converging to zero as $k \to \infty$



Gradient Descent

- Second, there are functions (even convex) with no minimum points, like $f(x) = \exp(-x)$, for which gradient descent with a positive learning rate converges to $+\infty$



Multi-dimensional Gradient Descent

- The extension to multi-dimensional case is straightforward.
- Let's assume $\theta \in \mathbb{R}^m$, so $f : \mathbb{R}^m \to \mathbb{R}$. The gradient of $f(\theta)$ with respect to θ is

$$\nabla_\theta f = \begin{bmatrix} \partial f / \partial \theta_1 \\ \partial f / \partial \theta_2 \\ \vdots \\ \partial f / \partial \theta_m \end{bmatrix}$$

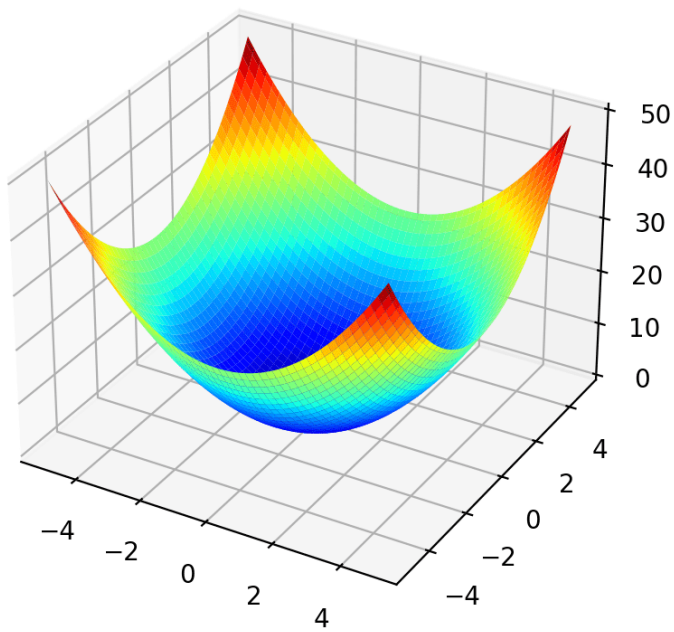
- The algorithm remains the same, except that the update step becomes

$$\theta^{(t)} = \theta^{(t-1)} - \eta \nabla_\theta f(\theta^{(t-1)})$$

- Any termination criteria that depended on the dimension of θ would have to change.

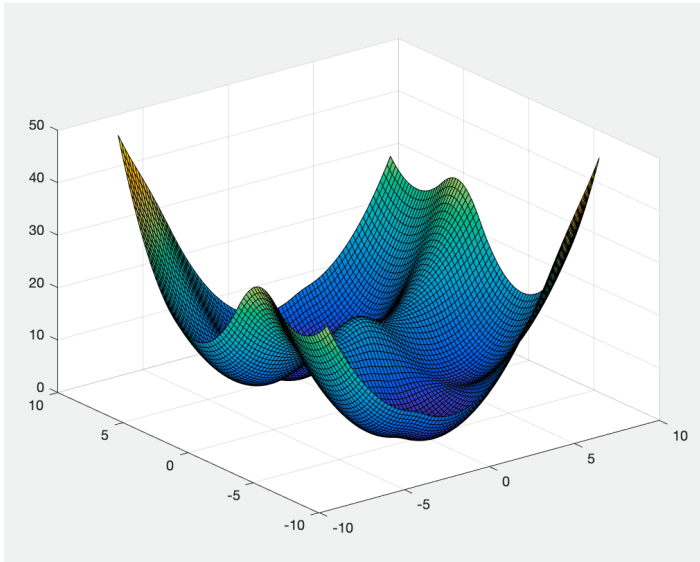
Multi-dimensional Gradient Descent

- Multi-dimensional convex ("bowl shaped") function



Multi-dimensional Gradient Descent

- Multi-dimensional non-convex function



Gradient Descent for Regression

- Recall that choosing a loss function is the first step in formulating a machine-learning problem as an optimization problem
- For regression, we studied the mean squared loss. This leads to the ordinary least squares objective

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \left(\boldsymbol{\theta}^{\mathrm{T}} \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

- We use the gradient of the objective with respect to the parameters,

$$\nabla_{\boldsymbol{\theta}} J = \frac{2}{n} \underbrace{\mathbf{X}^{\mathrm{T}}}_{d \times n} \underbrace{(\mathbf{X}\boldsymbol{\theta} - \mathbf{Y})}_{n \times 1}$$

Gradient Descent for Regression

- For regression problems, the update equation now becomes

$$\theta^{(t)} = \theta^{(t-1)} - \eta \nabla_{\theta} J$$

- Gradient descent could also be applied to numerically compute a solution, using the update rule

$$\theta^{(t)} = \theta^{(t-1)} - \eta \frac{2}{n} \sum_{i=1}^{n} \left(\left[\theta^{(t-1)} \right]^{\mathrm{T}} \mathbf{x}^{(i)} - y^{(i)} \right) \mathbf{x}^{(i)} .$$

Python Implementation of Gradient Descent

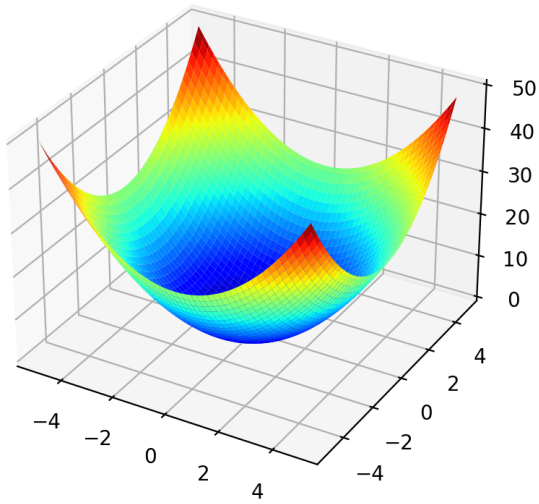
```python
def gradient_descent(X, y, learning_rate=0.01, num_iterations=1000):
    """ Performs gradient descent to learn w.  """
    N, M = X.shape
    w = np.zeros(M) # Initialize w to zeros
    cost_history = []
    for i in range(num_iterations):
        y_pred = np.dot(X, w) # Compute the hypothesis/predictions
        errors = y_pred - y   # Compute the residuals/errors
        cost = (1 / N) * np.dot(errors, errors) # Compute the cost (MSE)
        cost_history.append(cost)
        gradient = (2 / N) * np.dot(X.T, errors) # Compute the gradient
        w -= learning_rate * gradient    # Update the parameters
        # Print cost every 100 iterations
        if (i+1) % 100 == 0:
            print(f"Iteration {i+1}: Cost {cost:.4f}")
    return w, cost_history
```

Stochastic Gradient Descent

- Recall the equation for the gradient is a sum

$$\nabla_\theta J = \frac{2}{n} \sum_{i=1}^{n} \left(\left[\theta^{(t-1)}\right]^{\mathrm{T}} \mathbf{x}^{(i)} - y^{(i)} \right) \mathbf{x}^{(i)}$$

- Rather than take a big step in the gradient direction, we can randomly select a term of the sum, and take a small step in that direction



Stochastic Gradient Descent

- Let's assume our objective has the form

$$f(\theta) = \sum_{i=1}^{n} f_i(\theta) \quad ,$$

- Here is pseudo-code for applying SGD to the objective f; it assumes we know the form of $\nabla_\theta f_i$ for all i in $1 \ldots n$:

Algorithm 3 Stochastic-Gradient-Descent

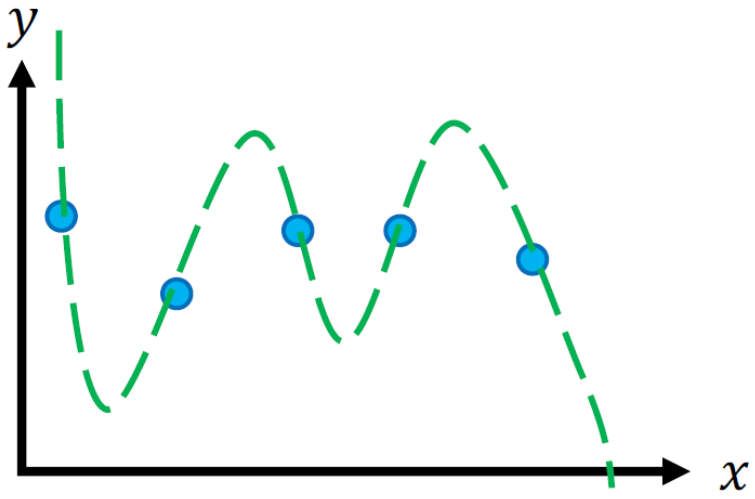
```
procedure STOCHASTIC-GRADIENT-DESCENT(θ_init, η, f, ∇_θ f_1, ..., ∇_θ f_n, T)
    θ^(0) ← θ_init
    for i = 1 to T do
        randomly select i ∈ {1, 2, ..., n}
        θ^(t) = θ^(t−1) − η ∇_θ f_i(θ^(t−1))
    end for
return θ^(t)
end procedure
```

Outline

- Introduction
- Linear Regression
- Gradient Descent
- Bias-Variance Tradeoff

Issues with Fitting

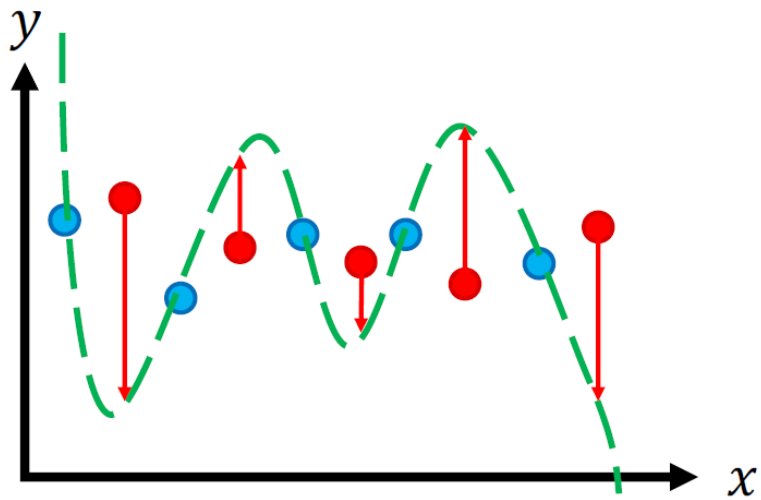
- By increasing model capacity, we can fit a dataset of any n points with a polynomial of degree n



Issues with Fitting

The goal in machine learning is prediction

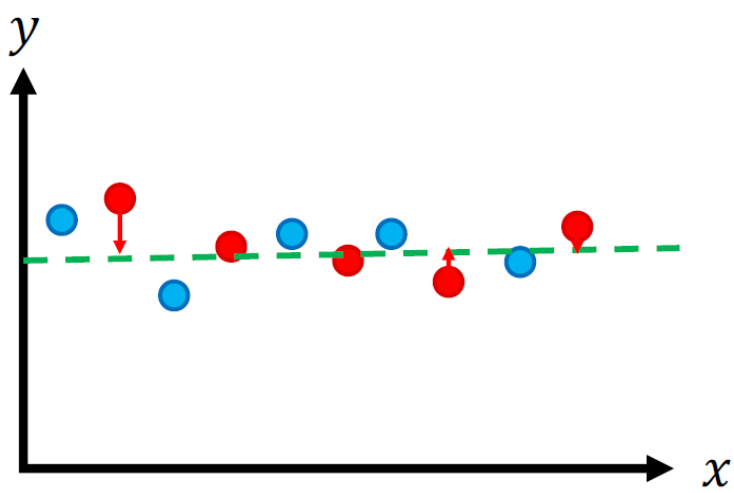
- Given a new input $\mathbf{x}$, predict the label y.
- The errors of new inputs is very large!



Issues with Fitting

The goal in machine learning is prediction

- Given a new input $\mathbf{x}$, predict the label y.
- A linear model actually works better!



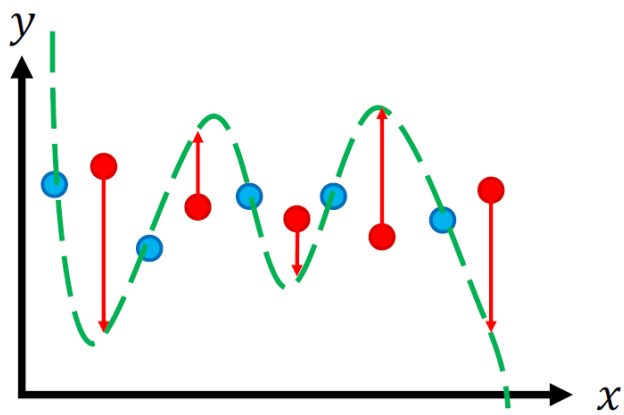
Overfitting and Underfitting

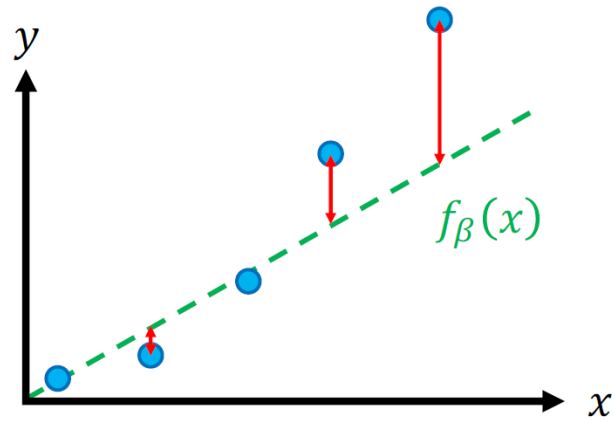
Overfitting

- Fit the training data well
- Fit new test data x, y poorly

Underfitting

- Fit the training data poorly
- May fit new test data poorly too





Overfitting and Underfitting

Why does overfitting happen?

- Overfitting typically due to fitting noise in the data
- Noise in labels
  - True data generating process is more complex than we can capture
  - May depend on unobserved features
- Noise in features
  - Measurement error in the feature values
  - Errors due to preprocessing
  - Some features might be irrelevant to the decision function

Overfitting and Underfitting

How to Fix Underfitting/Overfitting?

- Choose the right model family!

Role of Capacity

- Capacity of a model family captures "complexity" of data it can fit
  - Higher capacity → likely to overfit (model family has high variance)
  - Lower capacity → likely to underfit (model family has high bias)
- For linear regression, capacity corresponds to feature dimension d, i.e., number of features

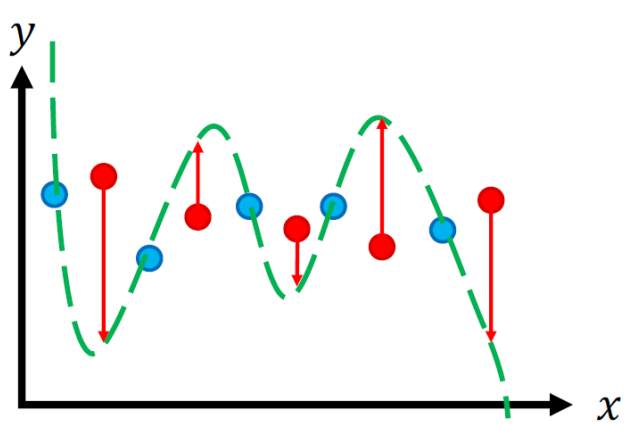
Bias-Variance Tradeoff

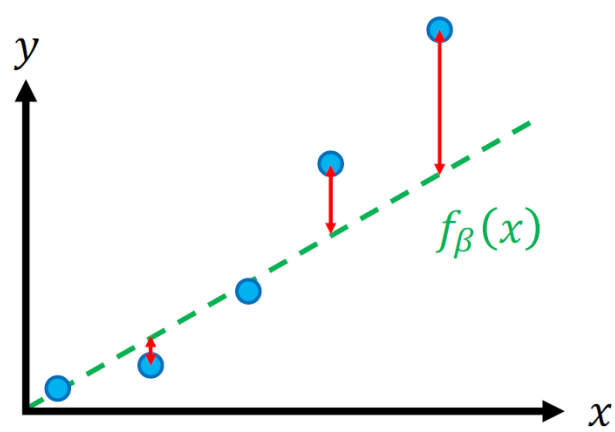
Overfitting (high variance)

- High capacity model capable of fitting complex data
- Insufficient data to constrain it

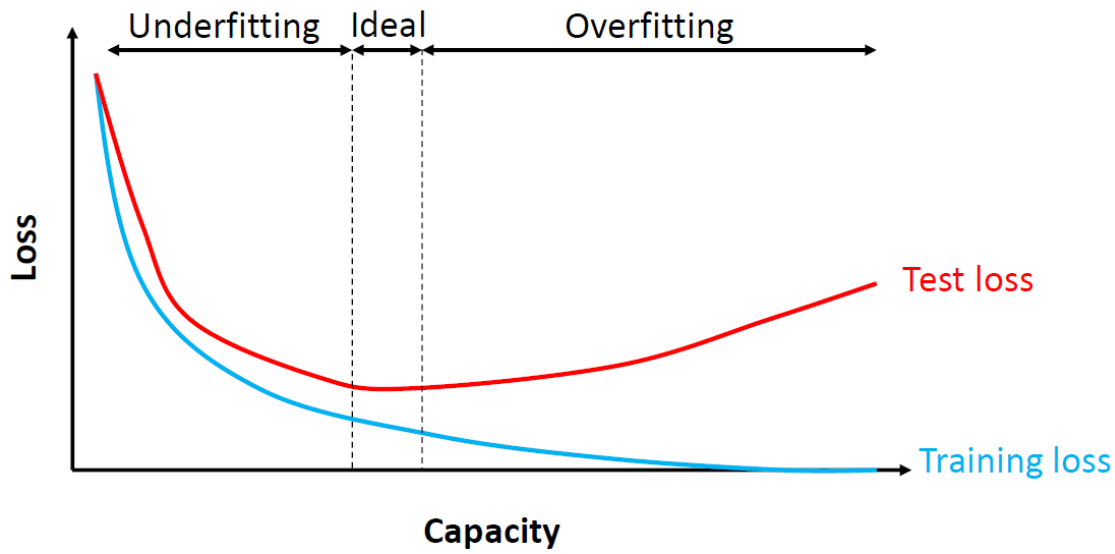
Underfitting (high bias)

- Low capacity model that can only fit simple data
- Sufficient data but poor fit





Bias-Variance Tradeoff



Bias-Variance Tradeoff

- For linear regression, increasing feature dimension d
  - Tends to increase capacity
  - Tends to decrease bias but increase variance
- Need to construct basis functions to balance tradeoff between bias and variance
  - Rule of thumb: $n \approx d \log d$
  - Large fraction of data science work is data cleaning + feature engineering

Bias-Variance Tradeoff

- Increasing number of data poinsts
  - Tends to increase bias and decrease variance
- General strategy
  - High bias: Increase model capacity
  - High variance: Increase data size n (i.e., gather more labeled data)